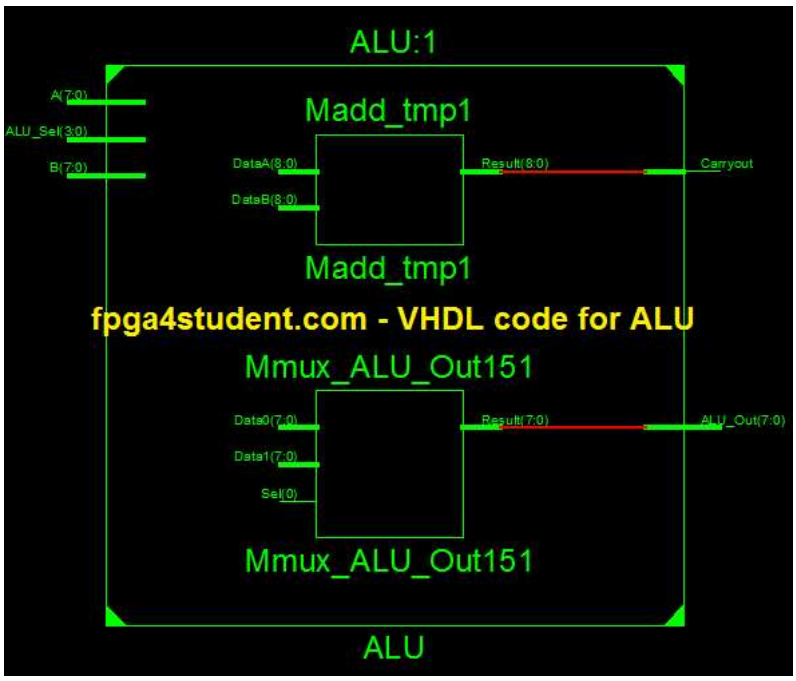
VHDL code for Arithmetic Logic Unit (ALU)

Arithmetic Logic Unit (ALU) is one of the most important digital logic components in CPUs. It normally executes logic and arithmetic operations such as addition, subtraction, multiplication, division, etc.

In this VHDL project, an ALU is designed and implemented in VHDL. VHDL code for the ALU is fully presented.



The logic and arithmetic operations being implemented in the ALU are as follows:

1. Arithmetic Addition
ALU_Out = A + B;
2. Arithmetic Subtraction
ALU_Out = A - B;
3. Arithmetic Multiplication
ALU_Out = A * B;
4. Arithmetic Division
ALU_Out = A / B;
5. Logical Shift Left
ALU_Out = A logical shifted left by 1;
6. Logical Shift Right
ALU_Out = A logical shifted right by 1;
7. Rotate Left
ALU_Out = A rotated left by 1;
8. Rotate Right
ALU_Out = A rotated right by 1;
9. Logical AND
ALU_Out = A AND B;
10. Logical OR
ALU_Out = A OR B;
11. Logical XOR
ALU_Out = A XOR B;
12. Logical NOR
ALU_Out = A NOR B;
13. Logical NAND
ALU_Out = A NAND B;

14. Logical XNOR
ALU_Out = A XNOR B;
15. Greater comparison
ALU_Out = 1 if A > B else 0;
16. Equal comparison
ALU_Out = 1 A = B else 0;
VHDL code for ALU:

```
-- fpga4student.com: FPGA Projects, Verilog projects, VHDL projects
-- VHDL code for ALU
-- @fpga4student
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use ieee.NUMERIC_STD.all;
-----------------------------------------------
---------- ALU 8-bit VHDL --------------------
-----------------------------------------------
entity ALU is
  generic (
     constant N: natural := 1  -- number of shited or rotated bits
    );

    Port (
    A, B     : in  STD_LOGIC_VECTOR(7 downto 0);  -- 2 inputs 8-bit
    ALU_Sel  : in  STD_LOGIC_VECTOR(3 downto 0);  -- 1 input 4-bit for selecting f
    ALU_Out   : out  STD_LOGIC_VECTOR(7 downto 0); -- 1 output 8-bit
    Carryout : out std_logic        -- Carryout flag
    );
end ALU;
architecture Behavioral of ALU is

signal ALU_Result : std_logic_vector (7 downto 0);
signal tmp: std_logic_vector (8 downto 0);

begin
   process(A,B,ALU_Sel)
 begin
  case(ALU_Sel) is
  when "0000" => -- Addition
   ALU_Result <= A + B ;
  when "0001" => -- Subtraction
   ALU_Result <= A - B ;
  when "0010" => -- Multiplication
   ALU_Result <= std_logic_vector(to_unsigned((to_integer(unsigned(A)) * to_intege
  when "0011" => -- Division
   ALU_Result <= std_logic_vector(to_unsigned(to_integer(unsigned(A)) / to_integer
  when "0100" => -- Logical shift left
   ALU_Result <= std_logic_vector(unsigned(A) sll N);
  when "0101" => -- Logical shift right
   ALU_Result <= std_logic_vector(unsigned(A) srl N);
  when "0110" => --  Rotate left
   ALU_Result <= std_logic_vector(unsigned(A) rol N);
  when "0111" => -- Rotate right
   ALU_Result <= std_logic_vector(unsigned(A) ror N);
  when "1000" => -- Logical and
   ALU_Result <= A and B;
  when "1001" => -- Logical or
   ALU_Result <= A or B;
  when "1010" => -- Logical xor
   ALU_Result <= A xor B;
  when "1011" => -- Logical nor
   ALU_Result <= A nor B;
```

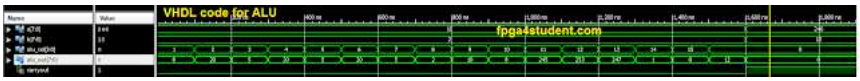
```
  when "1100" => -- Logical nand
   ALU_Result <= A nand B;
  when "1101" => -- Logical xnor
   ALU_Result <= A xnor B;
  when "1110" => -- Greater comparison
   if(A>B) then
    ALU_Result <= x"01" ;
   else
    ALU_Result <= x"00" ;
   end if;
  when "1111" => -- Equal comparison
   if(A=B) then
    ALU_Result <= x"01" ;
   else
    ALU_Result <= x"00" ;
   end if;
  when others => ALU_Result <= A + B ;
  end case;
 end process;
 ALU_Out <= ALU_Result; -- ALU out
 tmp <= ('0' & A) + ('0' & B);
 Carryout <= tmp(8); -- Carryout flag
end Behavioral;
```

Testbench VHDL code for ALU:

```
-- fpga4student.com: FPGA Projects, Verilog projects, VHDL projects
-- Testbench VHDL code for ALU
-- @fpga4student
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
use IEEE.std_logic_unsigned.all;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--USE ieee.numeric_std.ALL;

ENTITY tb_ALU IS
END tb_ALU;

ARCHITECTURE behavior OF tb_ALU IS

    -- Component Declaration for the Unit Under Test (UUT)

    COMPONENT ALU
    PORT(
         A : IN  std_logic_vector(7 downto 0);
         B : IN  std_logic_vector(7 downto 0);
         ALU_Sel : IN  std_logic_vector(3 downto 0);
         ALU_Out : OUT  std_logic_vector(7 downto 0);
         Carryout : OUT  std_logic
        );
    END COMPONENT;
       --Inputs
   signal A : std_logic_vector(7 downto 0) := (others => '0');
   signal B : std_logic_vector(7 downto 0) := (others => '0');
   signal ALU_Sel : std_logic_vector(3 downto 0) := (others => '0');

  --Outputs
   signal ALU_Out : std_logic_vector(7 downto 0);
   signal Carryout : std_logic;

 signal i:integer;
BEGIN
```

```
 -- Instantiate the Unit Under Test (UUT)
   uut: ALU PORT MAP (
          A => A,
          B => B,
          ALU_Sel => ALU_Sel,
          ALU_Out => ALU_Out,
          Carryout => Carryout
        );



   -- Stimulus process
   stim_proc: process
   begin
      -- hold reset state for 100 ns.
      A <= x"0A";
  B <= x"02";
  ALU_Sel <= x"0";

  for i in 0 to 15 loop
   ALU_Sel <= ALU_Sel + x"1";
   wait for 100 ns;
  end loop;
      A <= x"F6";
  B <= x"0A";
      wait;
   end process;

END;
```

Simulation waveform for the ALU:



Recommended VHDL projects:

1. What is an FPGA? How VHDL works on FPGA
2. VHDL code for FIFO memory
3. VHDL code for FIR Filter
4. VHDL code for 8-bit Microcontroller
5. VHDL code for Matrix Multiplication
6. VHDL code for Switch Tail Ring Counter
7. VHDL code for digital alarm clock on FPGA
8. VHDL code for 8-bit Comparator
9. How to load a text file into FPGA using VHDL
10. VHDL code for D Flip Flop
11. VHDL code for Full Adder
12. PWM Generator in VHDL with Variable Duty Cycle
13. VHDL code for ALU
14. VHDL code for counters with testbench
15. VHDL code for 16-bit ALU
16. Shifter Design in VHDL
17. Non-linear Lookup Table Implementation in VHDL
18. Cryptographic Coprocessor Design in VHDL
19. Verilog vs VHDL: Explain by Examples
20. VHDL Code for Clock Divider on FPGA
21. How to generate a clock enable signal instead of creating another clock domain
22. VHDL code for debouncing buttons on FPGA
23. VHDL code for Traffic light controller
24. VHDL code for a simple 2-bit comparator
22. VHDL code for debouncing buttons on FPGA

23. VHDL code for Traffic light controller
24. VHDL code for a simple 2-bit comparator
25. VHDL code for a single-port RAM
26. VHDL code for Car Parking System using FSM
27. VHDL coding vs Software Programming
28. VHDL code for MIPS Processor
29. VHDL code for Moore FSM Sequence Detector
30. VHDL code for Seven-Segment Display on Basys 3 FPGA

DMCA PROTECTED

Privacy Policy | Disclaimer | Sitemap | Contact | Advertise Here
Copyright © 2016-2020 FPGA4student.com All Rights Reserved.